

CO5 – AT3: Mock Interview

Name:A.Rakesh

Reg No:192525388

Course: Database Management Systems (CSA0504)

Q1. Steps of Query Processing & How the Optimizer Chooses a Plan (BL3)

Query processing converts a declarative SQL query into an efficient sequence of low-level operations. It happens in four main stages:

1. Parsing & Translation — the parser checks the query's syntax and validates table/column names against the system catalog, then translates the SQL into an internal representation such as a relational-algebra expression or a parse/query tree.
2. Optimization — the optimizer generates several equivalent execution plans for the same query (different join orders, join algorithms, access paths) and estimates the cost of each using statistics from the catalog (table sizes, index availability, selectivity of predicates).
3. Evaluation Plan Selection & Code Generation — the plan with the lowest estimated cost is chosen; the system generates the actual low-level code (or an execution/query plan) that the query-evaluation engine will run.
4. Execution — the query-evaluation engine executes the chosen plan against the database and returns the result to the user/application.

On how the optimizer decides the best plan: it enumerates candidate plans (varying join order, join algorithm — nested-loop, sort-merge, hash — and access method — full scan vs. index scan), estimates the I/O and CPU cost of each using cardinality/selectivity statistics, and picks the plan with the minimum estimated total cost. For simple queries it may also apply heuristic rules (like pushing selections down) before costing, to prune the search space so that not every possible plan needs to be fully costed.

Q2. Heuristic vs Cost-Based Optimization (BL4)

Aspect	Heuristic Optimization	Cost-Based Optimization
Basis	Fixed transformation rules (rewrite rules)	Estimated I/O/CPU cost using statistics
Statistics needed	None	Table sizes, selectivity, index info from catalog
Speed of optimization	Fast — rules applied directly	Slower — must generate & cost multiple plans
Guarantee	Improves the plan, but not guaranteed optimal	Chooses the lowest-cost plan among those considered
Typical use	Simple queries; early rewrite phase	Complex queries with joins on large tables

In practice these are not either/or — most real optimizers use both together: heuristic rules (pushing selections/projections down, combining Cartesian products into joins) are applied first to reduce the search space and cut obviously wasteful plans, and then cost-based estimation is used to choose among the remaining, more comparable candidate plans — especially for join ordering, where the number of possible orders grows very fast and needs cost comparison rather than a fixed rule. I'd use pure heuristics

for a lightweight embedded system with no statistics collection, and cost-based optimization whenever accurate table/index statistics are available and the query involves multiple joins on large tables.

Q3. Database Connectivity in a Real Project — JDBC vs ODBC (BL3)

In a real project, the application layer never talks to the database engine directly — it goes through a standard connectivity API that abstracts away the vendor-specific protocol. In a Java-based project I would use JDBC: obtain a Connection via the DriverManager (or a connection pool such as HikariCP in production), build a PreparedStatement with placeholders for parameters, execute it, and process the ResultSet, closing all three resources afterward (ideally with try-with-resources).

Aspect	JDBC	ODBC
Full form	Java Database Connectivity	Open Database Connectivity
Language binding	Java-specific API	Language-independent (C-based API); usable from C, C++, and via bridges from other languages
Driver architecture	Pure Java drivers (Type 4 most common) talk directly to the DB over its native protocol	Native C drivers plus an ODBC Driver Manager that mediates between application and DB driver
Platform dependency	Platform-independent — "write once, run anywhere" (JVM)	Often platform/OS-dependent, tied to native driver binaries
Typical use case	Java web/enterprise applications, Android	Legacy or non-Java applications (e.g. C/C++ desktop apps), and as a bridge (JDBC-ODBC bridge) for databases without a native JDBC driver

I'd choose JDBC by default for any Java-based system for its type safety, portability, and connection-pooling ecosystem, and reach for ODBC only when integrating with legacy tools, non-JVM languages, or databases that expose an ODBC driver but no native JDBC driver.

Q4. ACID Properties with Real-World Examples (BL3)

Property	Meaning	Real-world example
Atomicity	A transaction executes fully or not at all — no partial updates	A fund transfer of Rs.5,000 from Account A to B: if the debit from A succeeds but the credit to B fails (e.g. system crash), the entire transaction is rolled back so A is not left short by Rs.5,000.
Consistency	A transaction moves the database from one valid state to another, preserving all defined constraints/rules	An airline seat-booking system must never allow two passengers to be assigned the same seat number on the same flight — every committed transaction must leave that constraint intact.
Isolation	Concurrent transactions do not interfere with each other's intermediate states	Two customers booking the last available concert ticket at the same time — isolation ensures only one transaction actually succeeds in decrementing the seat count to zero; the other sees it as sold out, rather than both seeing 1 seat available and overselling.
Durability	Once a transaction commits, its changes survive even a subsequent system crash	After an ATM withdrawal is confirmed and cash dispensed, even if the bank's server crashes one second later, the debited balance must still be reflected when the system comes back up — the customer's

		account correctly shows the reduced balance.
--	--	--

Q5. Concurrency Problems — Dirty Read, Lost Update, Unrepeatable Read — and How 2PL Solves Them (BL4)

The problems

- **Dirty Read:** a transaction reads a value written by another transaction that has not yet committed; if that other transaction later aborts, the first transaction has used a value that never really existed. Example: T1 updates a balance but hasn't committed; T2 reads that updated value; T1 rolls back — T2's read is now based on data that no longer exists.
- **Lost Update:** two transactions read the same item, then both write back updated values based on the value they read — the second write overwrites (loses) the first. Example: T1 and T2 both read stock = 10, each decrements by 1 and writes 9, instead of the correct 8.
- **Unrepeatable Read (non-repeatable read):** a transaction reads the same item twice and gets different values because another transaction updated and committed it in between. Example: T1 reads balance = 1000, T2 updates and commits balance = 1500, T1 re-reads within the same transaction and now sees 1500.

How Two-Phase Locking (2PL) solves them

1. 2PL requires a transaction to acquire all the locks it needs (growing phase) before releasing any (shrinking phase). This ordering prevents interleavings that create the above anomalies as long as locks are honored strictly (shared locks for reads, exclusive locks for writes).
2. Dirty reads are prevented in Strict 2PL specifically, because exclusive locks are held until commit — so no other transaction can even acquire a shared lock to read an uncommitted write.
3. Lost updates are prevented because a transaction must hold an exclusive lock on an item to write it, and 2PL forces a second transaction wanting to write the same item to wait until the first releases its lock — so the two writes can no longer interleave arbitrarily.
4. Unrepeatable reads are prevented because, once T1 acquires a shared lock on an item, 2PL prevents T2 from acquiring an exclusive lock (and thus updating it) until T1's shrinking phase releases the shared lock — so the value cannot change between T1's two reads within the same transaction.

Note: basic 2PL alone guarantees conflict-serializability and thus prevents lost updates and unrepeatable reads, but only Strict (or Rigorous) 2PL — which holds locks until commit/abort — fully eliminates dirty reads, since basic 2PL still permits releasing a write lock before commit.

Q6. Deadlock — Detection Algorithm and Prevention Strategies (BL4)

A deadlock occurs when two or more transactions are each waiting for a resource (lock) held by another in the set, forming a circular wait so that none can proceed.

Deadlock detection algorithm — Wait-For Graph

1. The DBMS maintains a wait-for graph where each node is a transaction; a directed edge T_i to T_j means T_i is waiting for a lock held by T_j .
2. Whenever a transaction blocks waiting for a lock, an edge is added to the graph.
3. A separate deadlock-detection process periodically (or on each new wait) searches the graph for a cycle, e.g. via depth-first search.

4. If a cycle is found (e.g. T1 to T2 to T1), the system declares a deadlock among the transactions on that cycle.
5. The system selects a victim transaction (using a criterion such as youngest transaction, least work done, or fewest locks held), aborts it, releases its locks, and the remaining transactions in the cycle can then proceed.

Two deadlock prevention strategies

- Wait-Die (non-preemptive, timestamp-based): an older transaction is allowed to wait for a younger one, but if a younger transaction requests a resource held by an older one, the younger transaction is aborted ("dies") and restarted later with its original timestamp.
- Wound-Wait (preemptive, timestamp-based): an older transaction "wounds" (forcibly aborts) a younger transaction holding a resource it needs, while a younger transaction must wait if the resource is held by an older one.

Both schemes only ever allow waiting in one direction based on transaction age, so a circular wait — and hence deadlock — can never form; this makes them prevention strategies rather than detection-and-recovery strategies. A third common approach is timeout-based prevention: if a transaction waits longer than a threshold, it is assumed deadlocked and rolled back, which is simple but can abort transactions that were not actually deadlocked.

Q7. Immediate Update vs Deferred Update Recovery (BL4)

Aspect	Immediate Update	Deferred Update
When DB is updated	Changes are written to the database as soon as they occur, even before commit	Changes are buffered and applied to the database only after the transaction commits
UNDO needed?	Yes — uncommitted changes may already be on disk, so they must be undone on rollback/crash	No — since nothing is written before commit, there is nothing to undo (this is the NO-UNDO/REDO scheme)
REDO needed?	Yes, for committed transactions whose writes may not have reached disk yet	Yes, for committed transactions whose writes were buffered but not yet flushed
Logging requirement	Must log both old value and new value (before-image + after-image) before the write, following write-ahead logging (WAL)	Only needs to log the new value (after-image), since old values on disk are never overwritten before commit
Recovery cost after crash	Higher — both UNDO and REDO passes needed	Lower — only a REDO pass needed
Memory/buffer overhead during execution	Lower — writes go to disk as they happen	Higher — all of a transaction's updates must be buffered until commit

When each is preferred: Immediate Update suits systems with limited memory where transactions cannot hold all their updates in a buffer until commit, or where updates need to be visible to the transaction itself (through the buffer pool) quickly; it is the more general-purpose, widely-used approach (used with Strict 2PL and write-ahead logging in most commercial databases). Deferred Update suits scenarios where transactions are short and their write-sets are small enough to buffer safely, and where minimizing recovery complexity/time (avoiding UNDO logic entirely) is a priority — e.g. simpler embedded or batch-processing systems.

Q8. Shadow Paging — Advantages and Limitations (BL3)

Shadow paging is a recovery technique that avoids logging entirely. The database is organized as a set of fixed-size pages accessed through a page table (an array mapping logical page numbers to physical disk locations).

How it works

1. Before a transaction starts, the current page table is copied to create a shadow page table; the original (current) page table continues to be used by the running transaction and becomes the working copy.
2. Whenever the transaction writes to a page, it does not overwrite the original page on disk — instead a new copy of the page is written to a free disk location, and the current page table is updated to point to this new copy. The shadow page table still points to the old, unmodified page.
3. If the transaction commits, the current page table (with all updated pointers) is made the new shadow page table — the switch is done by updating a single pointer, so commit is effectively atomic.
4. If the transaction aborts or the system crashes before commit, the system simply discards the current page table and reverts to the shadow page table, which still points to the original, unmodified pages — no undo is needed since the old data was never overwritten.

Advantages

- No UNDO or REDO logging is required at all — recovery is simply a matter of restoring the shadow page table, making crash recovery very fast.
- Commit is atomic and near-instantaneous — it only involves updating a page-table pointer, not flushing log records.
- Simpler recovery-manager logic since there is no log to parse or apply.

Limitations

- Data fragmentation: modified pages are scattered across disk rather than staying contiguous, hurting locality and read performance over time (a form of "garbage" old pages also accumulates and needs periodic reclamation).
- Poor support for concurrent transactions: maintaining separate shadow page tables per concurrent transaction, and merging them correctly at commit, is complex — most designs favor one transaction updating the page table at a time, which limits concurrency compared to log-based recovery combined with 2PL.
- Commit overhead grows with database size, since even a small update can require copying/updating higher levels of the page-table tree (if page tables are multi-level).
- Difficult to support features common in modern DBMSs, such as fine-grained rollback of a single transaction among many concurrently running ones, which log-based (WAL) recovery handles naturally.

In short, shadow paging trades simpler, log-free recovery for weaker concurrency and disk-locality characteristics — which is why virtually all modern commercial DBMSs use log-based (write-ahead logging) recovery instead.

Q9. Serializability and Testing Conflict-Serializability via Precedence Graph (BL4)

A schedule of concurrently executing transactions is serializable if its effect on the database is equivalent to that of some serial (one-transaction-at-a-time) execution of the same transactions. Serializability is the correctness criterion the concurrency-control manager tries to guarantee, since serial schedules are

trivially correct but running transactions strictly one at a time is too slow — we want interleaved (concurrent) schedules that are still equivalent to some serial order.

Conflict-serializability

A weaker, easier-to-test form is conflict-serializability: a schedule is conflict-serializable if it can be transformed into a serial schedule by repeatedly swapping adjacent, non-conflicting operations (operations from different transactions on the same data item, where at least one is a write, are conflicting and cannot be swapped).

Testing with a precedence graph

1. Create one node for each transaction in the schedule.
2. For every pair of conflicting operations (different transactions, same data item, at least one write) where the operation of T_i occurs before the conflicting operation of T_j in the schedule, draw a directed edge T_i to T_j .
3. After considering all conflicting pairs, examine the resulting graph.
4. If the graph has NO cycle, the schedule is conflict-serializable, and a valid serial order can be obtained via a topological sort of the graph.
5. If the graph HAS a cycle (e.g. T_i to T_j to T_i), the schedule is NOT conflict-serializable — no equivalent serial order exists.

This is exactly the technique applied earlier in Q4 of the constraint-based problem-solving assessment, where a cycle T_1 to T_2 to T_1 was found, proving that schedule was not conflict-serializable.

Q10. Two-Phase Locking Variations — Basic, Strict, Rigorous (BL4)

Variant	Rule	Prevents dirty reads?	Recoverable / Cascadeless?
Basic 2PL	Growing phase acquires all locks; shrinking phase releases locks — but locks can start being released as soon as the transaction no longer needs them, even before commit	No	Guarantees serializability only; may allow cascading rollbacks
Strict 2PL	Same growing phase, but ALL exclusive (write) locks are held until commit/abort	Yes	Recoverable and cascadeless
Rigorous 2PL	ALL locks — both shared and exclusive — are held until commit/abort (no early release of any lock)	Yes	Recoverable, cascadeless, and additionally guarantees the schedule is equivalent to a serial order matching commit order

Which is most commonly used and why: Strict 2PL is the most widely used in practice. Basic 2PL guarantees serializability but still permits dirty reads and cascading rollbacks (if a transaction releases a write lock and aborts after another transaction has already read that dirty value), which is unacceptable for reliable systems. Rigorous 2PL is safest of all (and easiest to reason about, since transactions appear to execute in their commit order) but is slightly more restrictive than necessary — it holds even shared/read locks longer than Strict 2PL requires, reducing concurrency a bit further. Strict 2PL is the practical sweet

spot: it eliminates dirty reads and cascading aborts (by holding write locks till commit) while still allowing a transaction to release shared/read locks a little earlier than Rigorous 2PL would, giving it slightly better concurrency — which is why most commercial database systems (e.g. under the SERIALIZABLE isolation level) implement Strict 2PL as their default locking protocol.